CIS 3300 · OVERVIEW

Lambda Expressions and Streams

Overview and objectives

- **Course:** CIS 3300, Intermediate Object-Oriented Programming (Java)
- **Text:** *Core Java, Volume I* (13th ed.), Section 6.2, and *Core Java, Volume II* (13th ed.), Chapter 1
- **Materials:** two parts of class notes, with every example tested, and the Speedrun Leaderboard guides for Checkpoints 5 and 6
- **Prerequisite you already have:** interfaces; generics and collections; and natural order with `Comparable`

1. The short version

Some methods need to be told *what to do*: sorting needs an order, and removing needs to know which elements to remove. A *lambda expression* writes that behavior in one short expression and passes it to a method, where it becomes an object of a *functional interface*. You will write lambdas and method references, build comparators from small pieces, and use the collection methods that take lambdas in place of whole loops.

Streams build on lambdas. A stream processes a collection as a pipeline of steps, such as "keep the long songs, take their titles, sort them," and the code reads like a description of the result. You will build pipelines, handle results that might be empty with `Optional`, group and summarize data with collectors, and learn when a plain loop is still the clearer choice.

2. What We Will Cover

- **Lambda expressions.** Passing behavior, functional interfaces (`Predicate`, `Function`, `Consumer`, `Supplier`, `BinaryOperator`, and `Comparator`), lambda syntax, the four kinds of method reference, effectively final variables, building comparators, and the collection and map methods that take lambdas.
- **Streams.** Pipelines and their parts, creating streams, intermediate and terminal operations, `Optional`, number streams, collectors for grouping and summarizing, laziness and single use, and the common pitfalls.

3. Objectives

By the end of this unit, you should be able to:

1. Explain what a functional interface is, and match `Predicate`, `Function`, `Consumer`, `Supplier`, `BinaryOperator`, and `Comparator` to the methods they declare.
2. Write lambda expressions with expression bodies and with block bodies.

3. Replace a lambda with an equivalent method reference, and recognize the four kinds of method reference.
4. Explain why a lambda can use only effectively final local variables.
5. Build comparators with `comparing`, `comparingInt`, `thenComparing`, and `reversed`.
6. Use `removeIf`, `forEach`, `replaceAll`, and `sort` on collections, and `forEach`, `merge`, and `computeIfAbsent` on maps.
7. Describe the parts of a stream pipeline, and explain that a stream never changes its source.
8. Use the intermediate operations `filter`, `map`, `sorted`, `distinct`, `limit`, and `skip`.
9. Use terminal operations, including `toList`, `count`, `anyMatch`, `min`, `max`, `findFirst`, and `reduce`.
10. Handle an `Optional` result safely with `orElse`, `map`, and `ifPresent`.
11. Sum, average, and summarize numbers with `mapToInt` and number streams.
12. Group and summarize data with `groupingBy` and its downstream collectors, and use `partitioningBy`, `joining`, and `toMap`.
13. Explain laziness and why a stream can be used only once.
14. Decide when a plain loop is clearer than a stream, including code that throws checked exceptions.

4. How to Show Up Ready

- **Read ahead.** Skim Core Java, Volume I, Section 6.2 for lambdas, and Volume II, Chapter 1 for streams.
- **Be comfortable with collections.** Every stream starts from a collection, and most end in one, so review the Generics and Collections notes first.
- **Find a loop to replace.** Pick a loop from one of your labs that filters, counts, or totals something, and try writing it as a pipeline once the notes have shown you how.
- **Run the examples yourself.** Type them in, change one step of a pipeline, and predict the output before you run it.
- **If you miss a class,** work through that day's Speedrun Leaderboard checkpoint guide from its starting project before the next class.